TDD Feature Specification & Architecture

Real-Time Collaborative Workspace (Angular Modern Architecture)

Angular Signals | Control Flow (@if/@for) | @defer Optimization | SSR & Hydration

ARCHITECTURE DECISION: SSR & HYDRATION SETUP

SSR Assessment for Real-Time Workspace

Should you use SSR? Yes, but with Client-Only Route Exceptions.

- **Landing & Board Preview Pages:** Render via SSR with Event Replay / Hydration for fast initial page load (FCP/LCP) and indexability.
- **Live Collaborative Workspace Canvas:** Disable SSR or bypass server hydration using client-side route guards / `isPlatformBrowser()` to prevent SSR overhead for WebSocket-heavy interactive canvases.

Angular CLI SSR Setup Requirements:

- **Add SSR package:** `ng add @angular/ssr`
- **Enable Hydration in `app.config.ts`:** Include `provideClientHydration(withEventReplay())`.
- **Build Pipeline:** Use Application Builder in `angular.json` (powered by Vite/Esbuild) with `"ssr": { "entry": "server.ts" }`.

PHASE 1: CORE SIGNAL ARCHITECTURE & BOARD SHELL

Feature 1.1: Signal-Based Dynamic Board & Column Layout

> *Goal: Lay down the standalone foundation and reactive layout state using Signals without legacy modules or change detection overhead.*

Implementation Tasks:

- Create `BoardService` utilizing modern Angular Signals (`signal()` , `computed()`) to store board layouts.
- Implement root routing using Standalone Components and Functional Route Guards.
- Utilize new template control flow (`@for` , `@if`) in layout components with explicit `track column.id` .

Acceptance Criteria (TDD):

```
Scenario: Board initialization
  Given the BoardService is instantiated
  When a board ID is loaded
  Then boardSignal() contains correct layout and totalCards() computed signal reflects accurate
card count

Scenario: Modern template tracking
  Given a list of 5 columns in state
  When a column is added or removed
  Then @for (column of columns(); track column.id) renders only mutated nodes
```

Feature 1.2: Fine-Grained Reactive Card Counter & Search Filter

> *Goal: Verify that derived state updates instantaneously using computed() signals without manual subscription management.*

Implementation Tasks:

- Implement search input using Signal inputs (`input()`) and bidirectional model signals (`model()`).
- Create a computed signal `filteredBoard` reacting to search query changes and structural board updates.

Acceptance Criteria (TDD):

```
Scenario: Filter reactivity
  Given search query signal is set to "Bug"
  When filteredBoard() computed signal is read
  Then it returns only cards where title or tag contains "Bug"

Scenario: Zero-subscription cleanup
  Given component is unmounted
  When underlying signals update
  Then no active subscriptions remain in memory
```

PHASE 2: CARD OPERATIONS & LINKEDSIGNAL

Feature 2.1: Card Detail Modal & Two-Way Signal Binding

> *Goal: Handle nested component state flow using modern Signal Input/Output primitives (input(), output(), model()).*

Implementation Tasks:

- Build `CardDetailComponent` as a standalone component.
- Replace `@Input()` and `@Output()` with signal inputs: `card = input.required<Card>()` and `cardChange = output<Card>()`.
- Use `linkedSignal()` to handle localized editing state that resets whenever the selected card changes.

Acceptance Criteria (TDD):

```
Scenario: Signal input propagation
  Given selected card passed via card signal input
  When parent updates selected card signal
  Then CardDetailComponent local linkedSignal automatically updates draft fields

Scenario: Draft isolation
  Given user edits card title in modal draft signal
  When user clicks "Cancel"
  Then global BoardService state remains untouched
```

PHASE 3 & 4: DEFERRED LOADING & REAL-TIME SYNC

Feature 3.1: Lazy-Loading Heavy Rich Text Editor & Comments

> *Goal: Defer heavy text editing and syntax highlighting packages until interaction to maintain initial performance.*

Implementation Tasks:

- Wrap Rich Text Comment Editor inside an `@defer` block with trigger conditions (`@defer (on interaction)`).
- Provide native loading and placeholder states using `@loading` and `@placeholder` blocks.

Acceptance Criteria (TDD):

```
Scenario: Chunk deferral
  Given initial page render
  When inspecting bundle chunks
  Then Rich Text Editor chunk is not loaded on initial hit

Scenario: Interaction trigger
  Given placeholder element @placeholder { <button>Add Comment</button> }
  When user clicks or focuses placeholder button
  Then @loading template displays briefly, followed by fully hydrated editor
```

Feature 4.1: Live WebSocket Integration via Interop Primitives

> *Goal: Bridge real-time WebSocket events into the Signal state tree seamlessly using toSignal() and toObservable().*

Implementation Tasks:

- Build `CollaborationService` using RxJS `WebSocketSubject` for real-time remote updates.
- Convert WebSocket streams into signals using `toSignal()` with proper initial values.
- Use `effect()` to sync local board modifications back through WebSocket when structural changes occur.

Acceptance Criteria (TDD):

```
Scenario: Inbound stream conversion
  Given active WebSocket connection
  When remote client emits CARD_MOVED event
  Then toSignal() stream bridge updates boardSignal() without manual subscribe()

Scenario: Effect trigger protection
  Given local user drags card to new column
  When signal state changes
  Then effect() transmits payload to WebSocket backend exactly once
```